

Servicio Nacional de Aprendizaje SENA

Regional Cauca C.T.P.I

Centro de Teleinformática y Producción Industrial

Competencia

DOM y eventos en JavaScript

Aprendiz

Lucero Valencia Bohórquez

Instructora

Yoly Alexandra Guevara Muñoz

Programa de formación

Analisis y Desarrollo de Software

FICHA: 3407734

2026

TALLER INVESTIGATIVO DOM Y EVENTOS EN JAVASCRIPT

Parte 1. Fundamentos del DOM

1. ¿Qué significa DOM y qué palabras componen esta sigla?

DOM significa Document Object Model, que en español se traduce como Modelo de Objetos del Documento. Es una forma de representar los documentos HTML para que JavaScript pueda acceder a ellos y modificarlos.

2. ¿Qué es el DOM y para qué sirve en el desarrollo web?

El DOM es una representación del documento HTML que crea el navegador cuando carga una página web. Sirve para que JavaScript pueda cambiar el contenido, los estilos y los elementos de la página, así como responder a las acciones del usuario.

Ejemplo: cambiar un título, mostrar un mensaje o modificar el color de un botón.

3. ¿Cómo representa el navegador un documento HTML mediante el DOM?

El navegador representa el documento HTML mediante una estructura en forma de árbol, donde cada elemento se convierte en un nodo. Los elementos principales contienen otros elementos, formando una relación entre padres e hijos.

Por ejemplo, el elemento body puede contener un título h1 y un párrafo p.

4. ¿Qué relación existe entre HTML, CSS, JavaScript y el DOM?

Cada uno cumple una función diferente en el desarrollo web:

- HTML: define la estructura y el contenido de la página.
- CSS: se encarga de los colores, tamaños y estilos visuales.
- JavaScript: permite agregar interactividad y modificar elementos.
- DOM: representa el documento y permite que JavaScript acceda a sus elementos y los modifique.

Ejemplo: HTML crea un botón, CSS le da color y JavaScript hace que aparezca un mensaje cuando el usuario lo presiona, utilizando el DOM para acceder al botón.

5. ¿Qué es un nodo en el DOM? Explique la diferencia entre un nodo de elemento, un nodo de texto y un nodo de documento.

Un nodo es cada una de las unidades que forman parte de la estructura del DOM.

Existen diferentes tipos:

- Nodo de elemento: representa una etiqueta HTML, como h1, p o body.

- Nodo de texto: representa el texto que se encuentra dentro de un elemento, como «Hola mundo».
- Nodo de documento: representa el documento completo y es el punto de entrada principal al DOM.

Por ejemplo, en <escape> Hola mundo </escape>, la etiqueta p es un nodo de elemento y el texto «Hola mundo» es un nodo de texto.

6. ¿Qué es el árbol DOM? Dibuje un árbol que represente la estructura HTML siguiente.

El árbol DOM es la representación jerárquica de los elementos de un documento HTML, organizados mediante relaciones de padres e hijos.

Código HTML:

```
<html>
  <body>
    <h1>Mi página</h1>
    <p>Hola mundo</p>
  </body>
</html>
```

7. ¿Qué ocurriría con una página web si JavaScript no pudiera acceder al DOM?

JavaScript no podría acceder directamente a los elementos HTML para modificarlos dinámicamente. Por ejemplo, no podría cambiar un título, actualizar un párrafo o responder mediante esos elementos a un clic del usuario.

Sin embargo, la página podría seguir mostrando su contenido y sus estilos HTML y CSS; lo que se limitaría sería la interactividad que depende del DOM.

8. ¿El DOM es lo mismo que el código HTML escrito por el desarrollador? Justifique su respuesta con un ejemplo.

No, el DOM no es lo mismo que el código HTML escrito por el desarrollador. El HTML es el código que se escribe para crear la estructura de la página, mientras que el DOM es la representación de esa estructura que construye el navegador.

<h1>Hola</h1>

El navegador crea un nodo de elemento h1 y un nodo de texto que contiene «Hola». Después, JavaScript puede modificar ese texto mediante el DOM sin necesidad de cambiar directamente el archivo HTML original.

Por lo tanto, el HTML define la estructura inicial y el DOM permite acceder a ella y modificarla mientras la página está funcionando.

Parte 2. Selección y manipulación de elementos del DOM

1. getElementById()

- ¿Cómo permite seleccionar un elemento por su identificador?

Permite buscar un elemento HTML utilizando su atributo id, que debe ser único dentro del documento. Devuelve el elemento encontrado o null si no existe.

2. getElementsByClassName()

- ¿Qué devuelve y cómo se diferencia de seleccionar un único elemento?

Permite seleccionar todos los elementos que tienen una clase determinada. Devuelve una colección llamada HTMLCollection, que puede contener varios elementos.

3. getElementsByTagName()

- ¿Cómo selecciona elementos por su etiqueta HTML?

Busca todos los elementos que tienen una etiqueta HTML específica, como p, h1 o div. Devuelve una colección HTMLCollection.

4. querySelector()

- ¿Cómo selecciona el primer elemento que coincide con un selector CSS?

Permite seleccionar el primer elemento que coincide con un selector CSS, como una clase, un identificador o una etiqueta. Devuelve el elemento encontrado o null si no hay coincidencias.

5. querySelectorAll()

- ¿Qué diferencia tiene con querySelector()?

querySelectorAll() selecciona todos los elementos que coinciden con un selector CSS y devuelve una colección NodeList. En cambio, querySelector() devuelve únicamente el primer elemento coincidente.

6. innerHTML

- ¿Cómo permite consultar o modificar el contenido HTML interno?

Permite leer o cambiar el contenido que está dentro de un elemento HTML. A diferencia de textContent, interpreta las etiquetas HTML que se asignan como contenido.

7. textContent

- ¿Cómo permite leer o cambiar el texto de un elemento?

Permite obtener o modificar el contenido textual de un elemento sin interpretar las etiquetas HTML como elementos.

8. style

- ¿Cómo se pueden modificar estilos CSS mediante JavaScript?

La propiedad style permite modificar directamente los estilos CSS de un elemento, como el color, el tamaño de la letra y el fondo.

9. classList

- ¿Para qué sirven add(), remove(), toggle() y contains()?

La propiedad classList permite administrar las clases CSS de un elemento.

- add(): agrega una clase.
- remove(): elimina una clase.
- toggle(): agrega una clase si no existe o la elimina si ya existe.
- contains(): comprueba si una clase está presente y devuelve true o false.

10. createElement()

- ¿Cómo se crea un elemento HTML desde JavaScript?

Permite crear un nuevo elemento HTML indicando el nombre de su etiqueta. El elemento creado todavía no aparece en la página hasta que se incorpora al documento.

11. appendChild()

- ¿Cómo se agrega un nodo a otro elemento?

Permite agregar un nodo al final de los elementos hijos de otro nodo. Si el nodo ya existe en el documento, lo mueve a la nueva ubicación.

12. remove()

- ¿Cómo se elimina un elemento del documento?

Permite eliminar un elemento del documento HTML y quitarlo de su posición dentro del DOM.

Parte 3. Investigación sobre eventos en JavaScript

3.1 Conceptos fundamentales

1. ¿Qué es un evento en JavaScript?

Un evento es una acción o suceso que ocurre en una página web, como hacer clic en un botón, escribir en un campo de texto o mover el mouse. JavaScript puede detectar estos eventos y ejecutar instrucciones cuando suceden.

Ejemplo: hacer clic en un botón para mostrar un mensaje.

2. ¿Por qué los eventos son importantes para crear páginas web interactivas?

Los eventos son importantes porque permiten que una página web responda a las acciones del usuario. Gracias a ellos, se pueden crear botones interactivos, formularios, menús desplegables y mensajes de confirmación.

3. ¿Qué relación existe entre un evento, un elemento HTML y una función de JavaScript?

El elemento HTML es donde ocurre la interacción, el evento es la acción que sucede y la función de JavaScript contiene las instrucciones que se ejecutan como respuesta.

Ejemplo: un botón es el elemento HTML, el clic es el evento y la función muestra un mensaje.

4. ¿Qué significa escuchar o detectar un evento?

Significa configurar JavaScript para que esté atento a una acción determinada y ejecute una función cuando esta ocurra.

Ejemplo: escuchar el evento click para detectar cuándo un usuario presiona un botón.

5. ¿Qué es un manejador de eventos (event handler)?

Es una función o instrucción que se ejecuta cuando ocurre un evento determinado. Por ejemplo, una función que muestra un mensaje cuando el usuario hace clic en un botón.

6. ¿Qué diferencia existe entre un evento y una función?

Un evento es una acción o suceso, como hacer clic en un botón. Una función es un bloque de código que realiza una tarea específica cuando se ejecuta.

Por ejemplo, el clic es el evento y la función que muestra «Hola» es la respuesta programada para ese evento.

15. ¿Qué es addEventListener() y cuál es su sintaxis general?

addEventListener() es un método de JavaScript que permite escuchar un evento en un elemento HTML y ejecutar una función cuando ocurre.

Sintaxis general:

elemento.addEventListener("evento", funcion);

Ejemplo:

```
let boton = document.getElementById("boton");

boton.addEventListener("click", function() {
   alert("¡Hiciste clic!");
});
```

Cuando el usuario hace clic en el botón, aparece una ventana con el mensaje «¡Hiciste clic!».

16. ¿Qué diferencia existe entre utilizar onclick directamente en HTML y utilizar addEventListener() desde JavaScript?

La diferencia principal es la forma de registrar el evento.

- onclick en HTML: se escribe directamente en la etiqueta HTML para indicar qué código se ejecuta al hacer clic.
- addEventListener() en JavaScript: permite registrar el evento desde el código JavaScript, manteniendo separadas la estructura HTML y la lógica del programa. Además, permite registrar varios manejadores para el mismo evento.

Ejemplo con onclick:

```
<button onclick="alert('Hola')">Presionar</button>
```

Ejemplo con addEventListener():

```
boton.addEventListener("click", function() {
   alert("Hola");
});
```

17. ¿Qué sucede si se registra más de un manejador para el mismo tipo de evento y elemento?

Si se registran varios manejadores con addEventListener() para el mismo tipo de evento y elemento, todos pueden ejecutarse cuando ocurre el evento, siguiendo el orden en que fueron registrados, siempre que ninguno sea eliminado o se interrumpa su ejecución.

Ejemplo:

```
boton.addEventListener("click", function() {
   console.log("Primer mensaje");
});

boton.addEventListener("click", function() {
   console.log("Segundo mensaje");
});
```

Al hacer clic en el botón, la consola muestra ambos mensajes.

18. ¿Qué es el objeto event y qué información puede proporcionar cuando ocurre una interacción?

El objeto event es un objeto que proporciona información sobre el evento que acaba de ocurrir. Puede indicar qué tipo de evento sucedió, qué elemento lo originó y otros datos relacionados con la interacción.

Por ejemplo, permite conocer el elemento que generó el evento mediante event.target y, en un evento de teclado, identificar la tecla presionada mediante event.key.

Ejemplo:

```
boton.addEventListener("click", function(event) {
   console.log("Tipo de evento:", event.type);
   console.log("Elemento presionado:", event.target);
});
```

Cuando el usuario hace clic, la consola muestra el tipo de evento (click) y el elemento HTML que lo originó.

3.2 Tipos eventos

Eventos principales en JavaScript

1. click

- ¿Cuándo se activa al pulsar un elemento?

Se activa cuando el usuario hace clic sobre un elemento, como un botón, un enlace o una imagen.

Ejemplo: boton.addEventListener("click", () => alert("Hola"));

2. dblclick

- ¿En qué se diferencia de click?

Se activa cuando el usuario hace doble clic rápidamente sobre un elemento. A diferencia de click, requiere dos clics consecutivos.

Ejemplo: elemento.addEventListener("dblclick", () => alert("Doble clic"));

3. mouseover

- ¿Cuándo se activa al situar el puntero sobre un elemento?

Se activa cuando el puntero del mouse entra en un elemento o en uno de sus elementos descendientes.

Ejemplo: caja.addEventListener("mouseover", () => caja.style.color = "red");

4. mouseout

- ¿Qué sucede cuando el puntero sale de un elemento?

Se activa cuando el puntero sale de un elemento o pasa de él a uno de sus elementos descendientes.

Ejemplo: caja.addEventListener("mouseout", () => caja.style.color = "black");

5. mouseenter

- ¿Cómo se comporta al entrar el puntero en un elemento?

Se activa cuando el puntero entra en el área de un elemento. A diferencia de mouseover, no se activa nuevamente al pasar entre sus elementos descendientes.

Ejemplo: caja.addEventListener("mouseenter", () => console.log("Puntero dentro"));

6. mouseleave

- ¿Cuándo se activa al abandonar un elemento?

Se activa cuando el puntero sale del área de un elemento, sin activarse por pasar entre sus elementos descendientes.

Ejemplo: caja.addEventListener("mouseleave", () => console.log("Puntero fuera"));

7. keydown

- ¿Qué ocurre cuando se presiona una tecla?

Se activa cuando el usuario presiona una tecla del teclado. Puede repetirse si mantiene la tecla presionada.

Ejemplo: document.addEventListener("keydown", () => console.log("Tecla presionada"));

8. keyup

- ¿En qué se diferencia de keydown?

Se activa cuando el usuario suelta una tecla. En cambio, keydown se activa al presionarla.

Ejemplo: document.addEventListener("keyup", () => console.log("Tecla soltada"));

9. input

- ¿Cuándo se activa al modificar el valor de un campo?

Se activa cuando el usuario modifica el valor de un campo de entrada, por ejemplo, al escribir o borrar texto.

Ejemplo: campo.addEventListener("input", () => console.log(campo.value));

10. change

- ¿Cuándo se detectan cambios en un campo de formulario?

Se activa cuando un campo registra un cambio confirmado en su valor. En un campo de texto normalmente ocurre cuando el usuario modifica el contenido y sale del campo; en una casilla de verificación, ocurre al cambiar su estado.

Ejemplo: campo.addEventListener("change", () => console.log("Valor cambiado"));

11. submit

- ¿Cómo se utiliza al enviar un formulario?

Se activa cuando se envía un formulario, por ejemplo, al presionar un botón de tipo submit. Permite validar o procesar los datos antes de enviarlos.

Ejemplo:

```
formulario.addEventListener("submit", function(event) {
    event.preventDefault();
    alert("Formulario enviado");
});
```

preventDefault() evita el envío predeterminado del formulario, por lo que el ejemplo muestra el mensaje sin recargar la página.

12. focus

- ¿Qué ocurre cuando un campo recibe el foco?

Se activa cuando un elemento, como un campo de texto, pasa a ser el elemento activo para la interacción del usuario.

Ejemplo: campo.addEventListener("focus", () => campo.style.backgroundColor = "yellow");

13. blur

- ¿Qué sucede cuando un campo pierde el foco?

Se activa cuando el usuario deja de interactuar directamente con un campo porque el foco pasa a otro elemento.

Ejemplo: campo.addEventListener("blur", () => campo.style.backgroundColor = "white");

14. load

- ¿Cuándo se utiliza para detectar la carga de un recurso o documento?

Se activa cuando un recurso ha terminado de cargarse. Por ejemplo, el evento load de window permite detectar cuándo se ha cargado completamente la página, incluidos sus recursos dependientes.

Ejemplo:

```
window.addEventListener("load", function() {
   console.log("La página terminó de cargar");
});
```

3.3 Preguntas de profundización

19. ¿Por qué un botón puede utilizar click, mientras que un campo de texto puede utilizar input?

Porque cada evento responde a una acción diferente. El evento click detecta cuando el usuario hace clic en un botón, mientras que input detecta cuando el usuario modifica el contenido de un campo de texto, por ejemplo, al escribir o borrar caracteres.

20. ¿Qué diferencia existe entre mouseenter y mouseover?

Ambos eventos se activan cuando el puntero del mouse entra en un elemento. La diferencia es que mouseenter no se activa al pasar entre los elementos hijos, mientras que mouseover sí puede activarse cuando el puntero entra en uno de esos elementos hijos.

21. ¿Qué diferencia existe entre keydown y keyup?

El evento keydown se activa cuando el usuario presiona una tecla del teclado. En cambio, keyup se activa cuando el usuario suelta la tecla.

Por ejemplo, al escribir una letra, keydown detecta la presión de la tecla y keyup detecta cuándo se deja de presionar.

22. ¿Por qué se utiliza preventDefault() al trabajar con determinados formularios?

Se utiliza para evitar el comportamiento predeterminado del navegador ante un evento. En los formularios, permite evitar que la página se recargue al enviarlos y así validar o procesar los datos mediante JavaScript.

Ejemplo:

```
formulario.addEventListener("submit", function(event) {
    event.preventDefault();
    alert("Formulario procesado");
});
```

23. ¿Qué sucede cuando se ejecuta event.target? Construya un ejemplo.

event.target permite identificar el elemento HTML donde se originó un evento.

Ejemplo:

```
<button id="miBoton">Presionar</button>

<script>
let boton = document.getElementById("miBoton");

boton.addEventListener("click", function(event) {
    console.log(event.target);
});
</script>
```

Cuando el usuario presiona el botón, la consola muestra el elemento HTML que originó el evento, en este caso, el botón.

24. ¿Qué diferencia existe entre event.target y event.currentTarget?

event.target identifica el elemento donde se originó el evento, mientras que event.currentTarget identifica el elemento al que está asociado el manejador que se está ejecutando.

Por ejemplo, si un botón contiene un texto dentro de un elemento span y se hace clic sobre ese texto, event.target puede ser el span, mientras que event.currentTarget será el botón si el manejador está registrado en él.

25. ¿Qué son la propagación de eventos, el bubbling y la captura?

La propagación de eventos es el proceso mediante el cual un evento se distribuye a través de los elementos relacionados del DOM.

- Bubbling (burbujeo): el evento se propaga desde el elemento donde se originó hacia sus elementos padres.
- Captura: el evento recorre la estructura desde los elementos superiores hacia el elemento donde se originó.

Por ejemplo, si un botón está dentro de un div, al hacer clic en el botón el evento puede llegar también al div mediante el burbujeo.

26. ¿En qué consiste la delegación de eventos y cuándo podría ser útil?

La delegación de eventos consiste en colocar un único manejador de eventos en un elemento padre para detectar eventos que ocurren en sus elementos hijos, aprovechando la propagación de eventos.

Es útil cuando una página tiene muchos elementos similares o cuando se agregan nuevos elementos dinámicamente, ya que evita tener que registrar un manejador individual para cada uno.

Ejemplo:

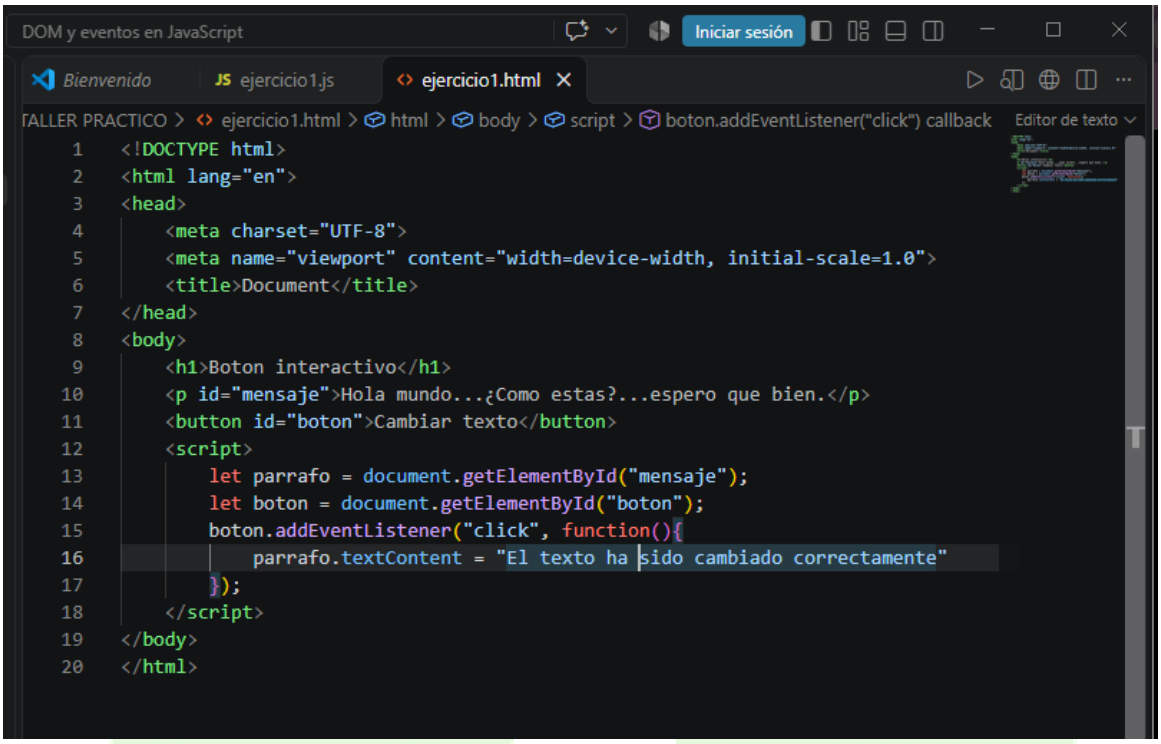
```
<ul id="lista">
   <li>Manzana</li>
   <li>Pera</li>
   <li>Banano</li>
</ul>

<script>
let lista = document.getElementById("lista");

lista.addEventListener("click", function(event) {
   if (event.target.tagName === "LI") {
      alert("Seleccionaste: " + event.target.textContent);
   }
});
</script>
```

Parte 4. Taller práctico de investigación

Ejercicio 1. Botón interactivo



1. ¿Cómo seleccionó el párrafo?

Utilicé document.getElementById("mensaje") para seleccionar el párrafo mediante su identificador id="mensaje".

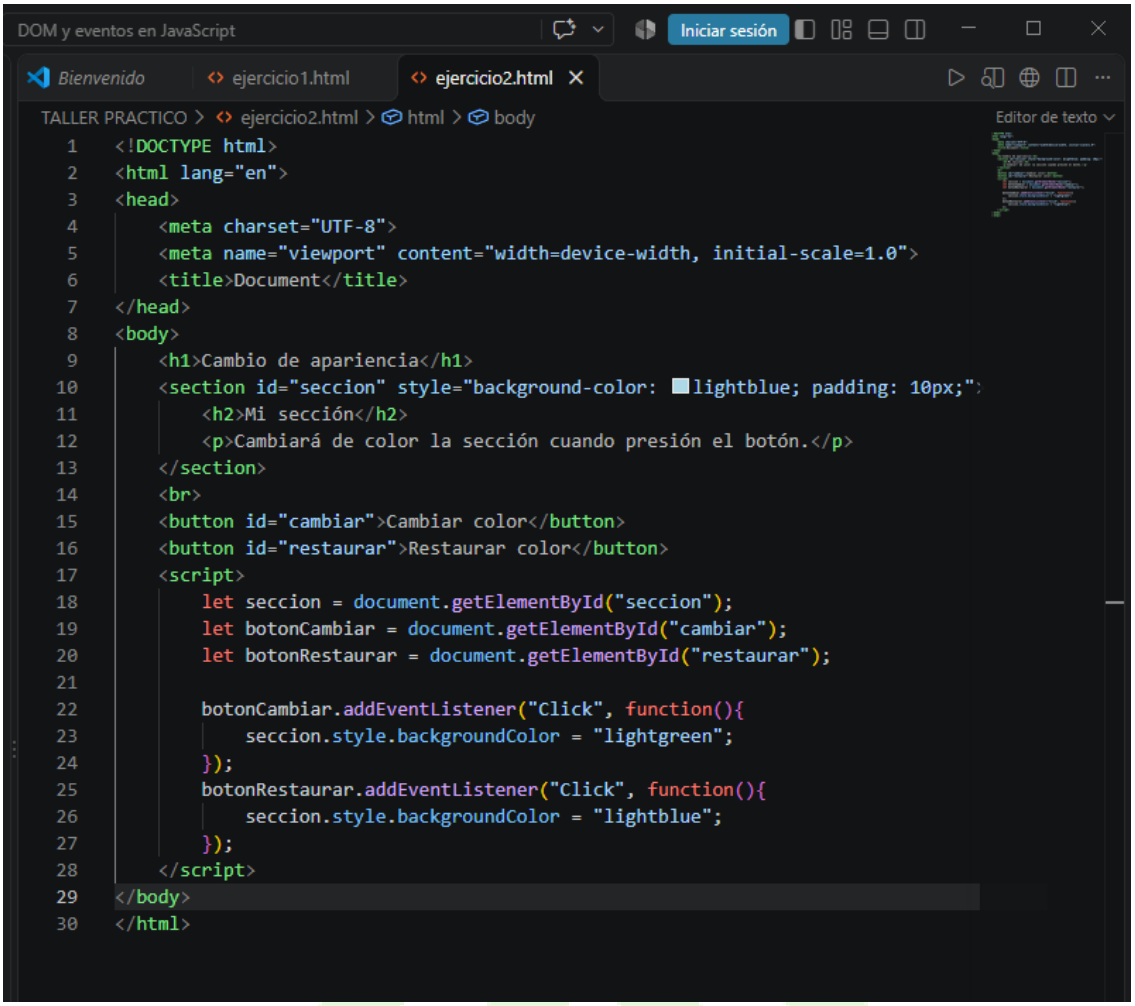
2. ¿Qué evento utilizó?

Utilicé el evento click mediante addEventListener("click", function() {}), que detecta cuando el usuario presiona el botón.

3. ¿Qué propiedad modificó?

Modifiqué la propiedad textContent, que permite cambiar el texto del párrafo sin modificar su etiqueta HTML.

Ejercicio 2. Cambio de apariencia

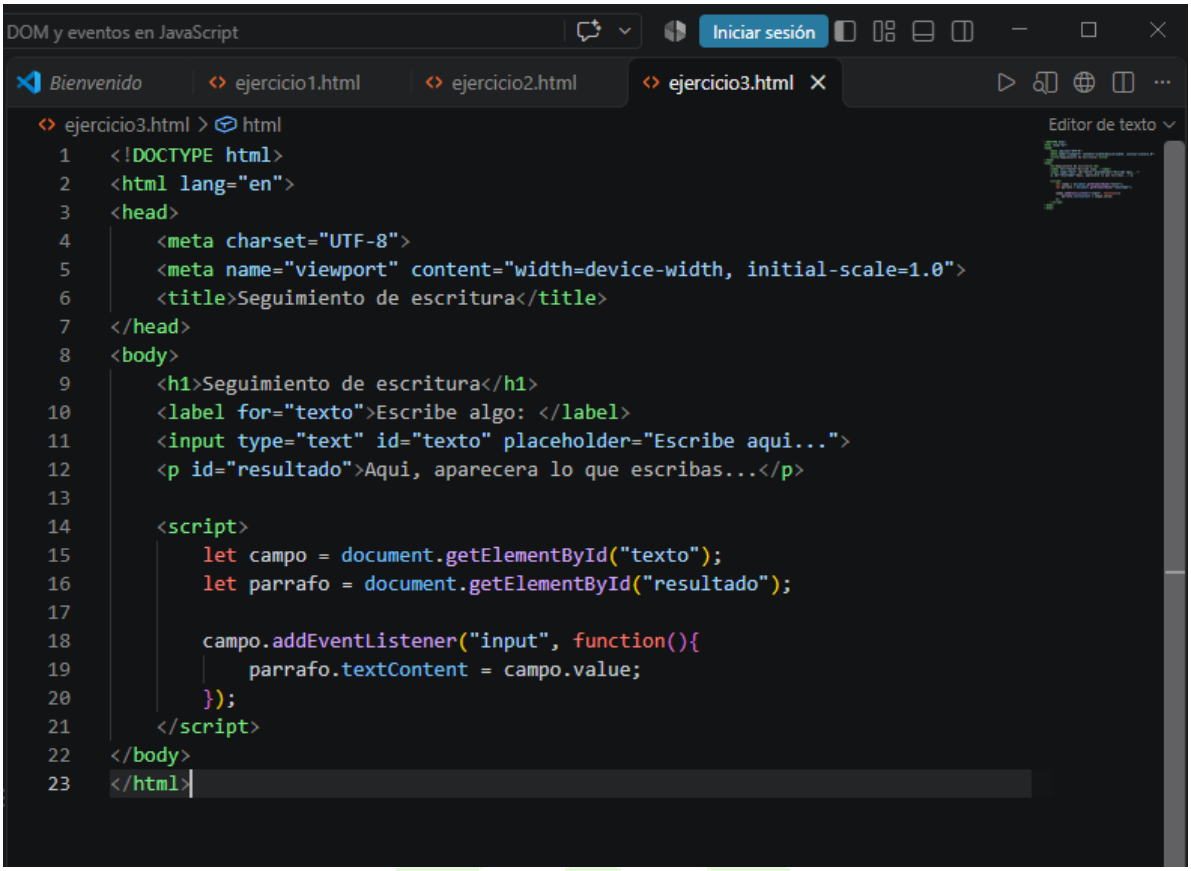


Es mejor utilizar clases CSS con classList cuando se quiere cambiar la apariencia de una página web, porque permite separar los estilos del código JavaScript y facilita el mantenimiento del programa.

Por ejemplo, con style se cambia directamente el color desde JavaScript, mientras que con classList se puede agregar o quitar una clase que contiene los estilos definidos en CSS.

Justificación: classList es más conveniente para manejar cambios de apariencia definidos en CSS, ya que permite reutilizar estilos y mantener el código organizado. Sin embargo, style también es útil cuando se necesita cambiar una propiedad CSS puntual de manera rápida y directa.

Ejercicio 3. Seguimiento de escritura



1. ¿Por qué es apropiado utilizar input en vez de click?

Es apropiado utilizar input porque detecta en tiempo real los cambios que el usuario realiza en un campo de texto, como escribir, borrar o modificar caracteres. En cambio, click detecta cuando el usuario hace clic sobre un elemento, pero no permite detectar por sí solo cada cambio en el texto.

2. ¿Qué información se obtiene de la propiedad value?

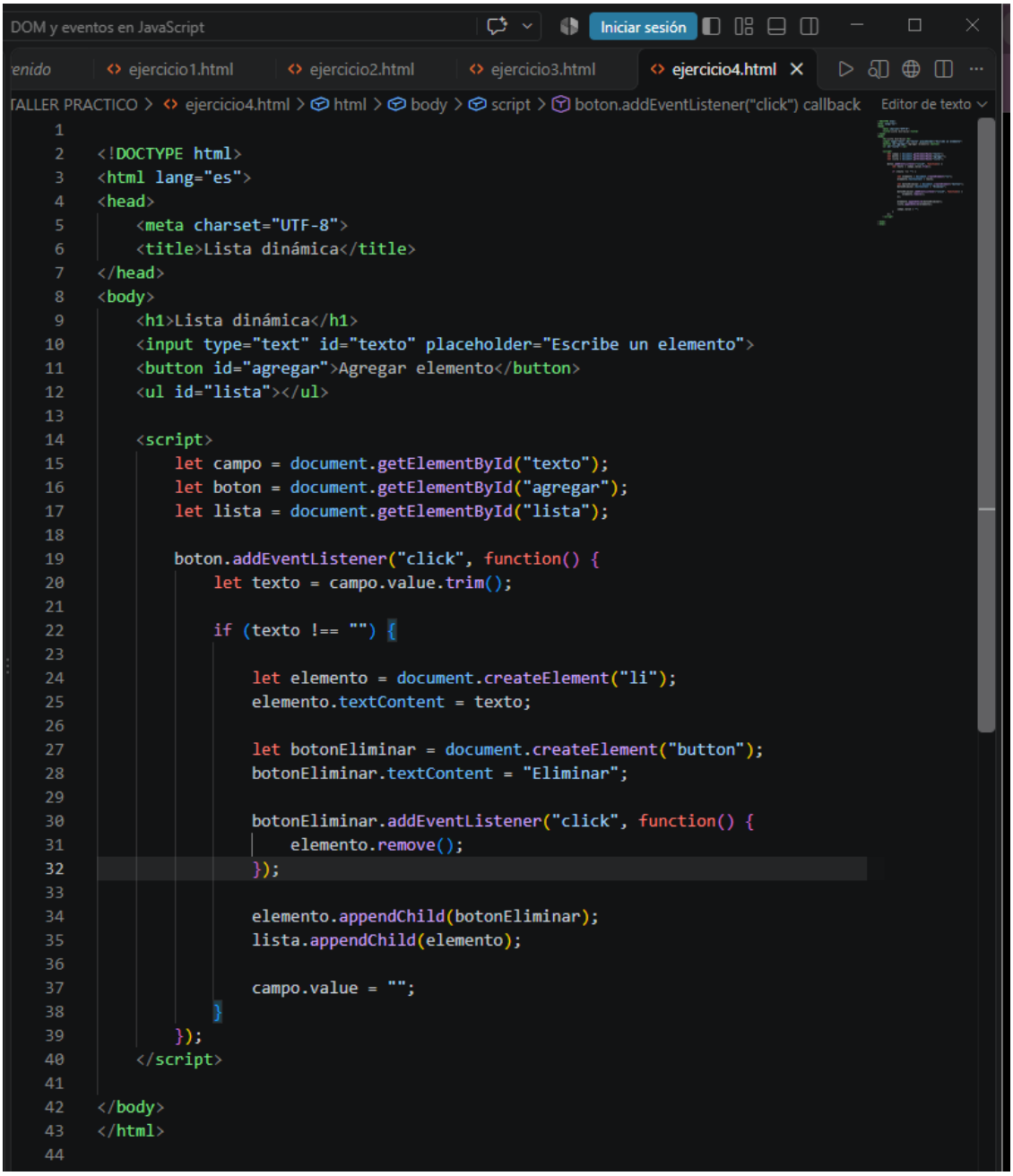
La propiedad value permite obtener el valor actual que el usuario ha escrito en un campo de entrada, como un nombre, una palabra o una frase. Esta información se puede utilizar para mostrar el texto en otro elemento o procesarlo con JavaScript.

Ejemplo:

```
campo.addEventListener("input", function() {
    parrafo.textContent = campo.value;
});
```

En este código, campo.value obtiene lo que el usuario escribe y parrafo.textContent lo muestra inmediatamente en el párrafo.

Ejercicio 4. Lista dinámica



1. ¿Para qué sirven createElement(), appendChild() y remove()?

- createElement(): sirve para crear un nuevo elemento HTML desde JavaScript. En el ejercicio, se utiliza para crear los elementos de la lista (li) y los botones de eliminar (button).
- appendChild(): sirve para agregar un elemento como hijo de otro elemento. En el ejercicio, permite agregar cada elemento a la lista y colocar su botón de eliminar dentro de él.
- remove(): sirve para eliminar un elemento del documento HTML. En el ejercicio, elimina el elemento de la lista seleccionado por el usuario.

2. ¿Cómo se asigna el evento al botón de eliminar?

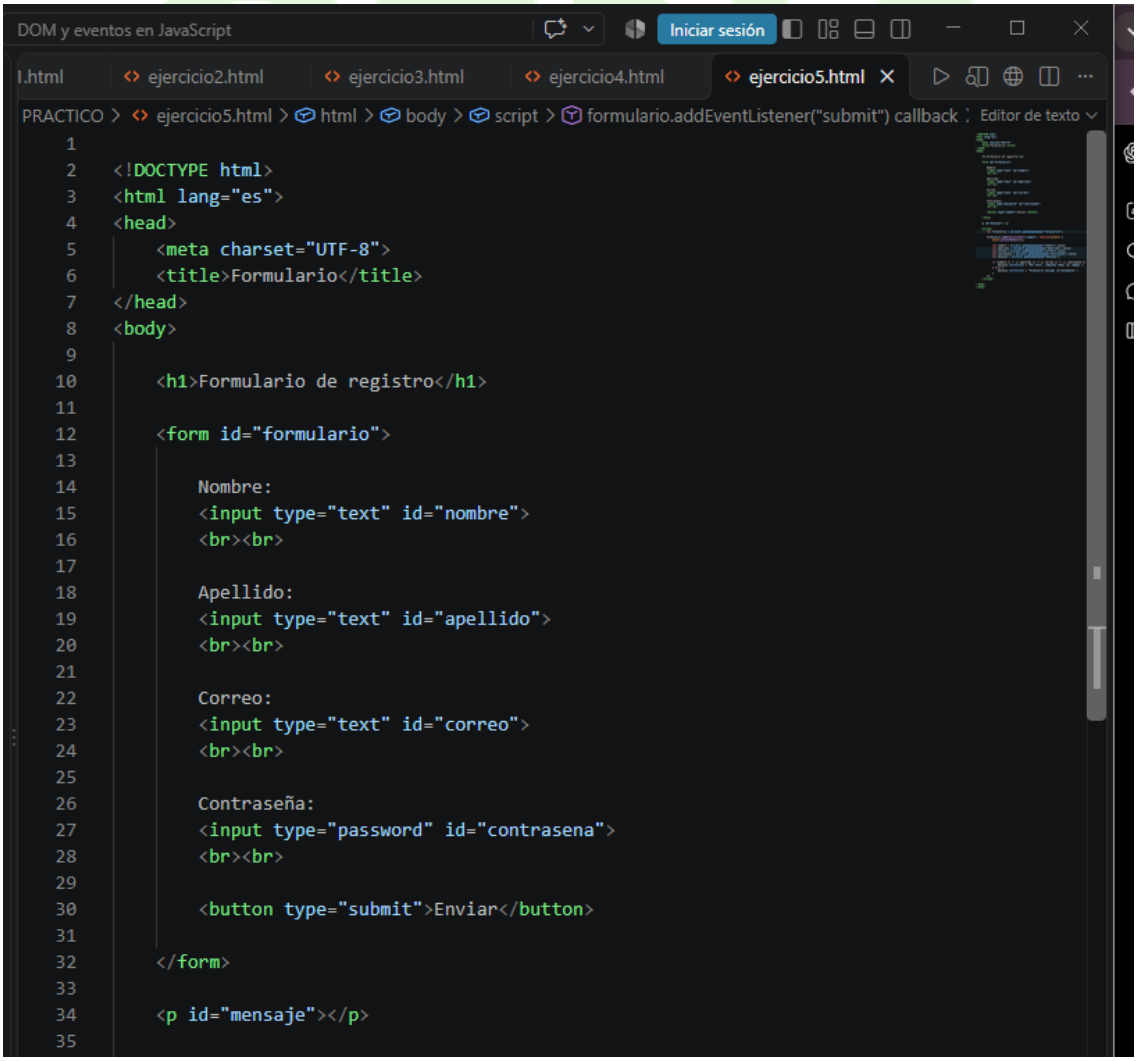
El evento se asigna mediante addEventListener("click", function() {}), que detecta cuándo el usuario hace clic en el botón y ejecuta la función correspondiente.

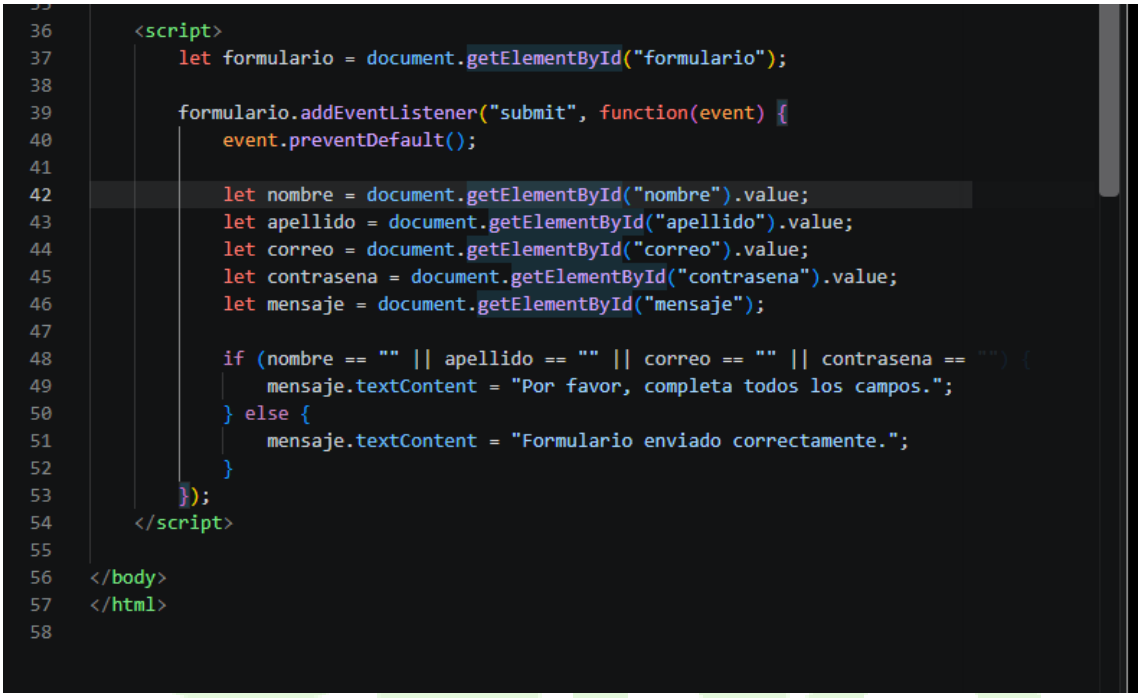
Ejemplo del código:

```
botonEliminar.addEventListener("click", function() {
    elemento.remove();
});
```

Explicación: cuando el usuario presiona el botón «Eliminar», se ejecuta la función y elemento.remove() elimina únicamente el elemento de la lista al que pertenece ese botón.

Ejercicio 5. Formulario con validación





```
    <script>
        let formulario = document.getElementById("formulario");

        formulario.addEventListener("submit", function(event) {
            event.preventDefault();

            let nombre = document.getElementById("nombre").value;
            let apellido = document.getElementById("apellido").value;
            let correo = document.getElementById("correo").value;
            let contrasena = document.getElementById("contrasena").value;
            let mensaje = document.getElementById("mensaje");

            if (nombre == "" || apellido == "" || correo == "" || contrasena == "") {
                mensaje.textContent = "Por favor, completa todos los campos.";
            } else {
                mensaje.textContent = "Formulario enviado correctamente.";
            }
        });
    </script>

</body>
</html>
```

1. ¿Cómo funciona submit?

submit es un evento que se activa cuando el usuario intenta enviar un formulario, por ejemplo, al hacer clic en el botón Enviar o al presionar Enter en un campo.

Ejemplo:

```
formulario.addEventListener("submit", function(event) {
    alert("El formulario se está enviando");
});
```

¿Qué hace cada parte?

- formulario: es el formulario que seleccionamos en HTML.
- addEventListener: permite escuchar un evento.
- "submit": indica que estamos escuchando el envío del formulario.
- function(event): contiene las instrucciones que se ejecutan cuando ocurre el evento.

2. ¿Para qué sirve preventDefault()?

preventDefault() sirve para evitar que el navegador realice la acción predeterminada de un evento.

En nuestro formulario, cuando hacemos clic en Enviar, normalmente el navegador intenta enviar los datos y puede recargar la página.

Por eso usamos:

```
formulario.addEventListener("submit", function(event) {
   event.preventDefault();

   alert("Formulario revisado");
});
```

La línea event.preventDefault() evita el envío normal del formulario y la recarga de la página, permitiéndonos revisar los datos con JavaScript primero.

Importante: preventDefault() no valida los campos por sí solo; únicamente evita la acción predeterminada.

3. ¿Cuál es la diferencia entre los atributos HTML de validación y la validación con JavaScript?

Validación en HTML	Validación con JavaScript
Es más sencilla de utilizar.	Permite crear reglas personalizadas.
Se configura mediante atributos.	Se programa mediante instrucciones.
El navegador realiza comprobaciones básicas.	Podemos comprobar condiciones más específicas.
Ejemplo: required.	Ejemplo: if (nombre == "").

Ejemplo de validación con HTML:

```
<input type="text" required>
<input type="email" required>
```

- required: obliga a completar el campo.
- type="email": comprueba que el contenido tenga un formato de correo reconocido por el navegador.

Ejemplo de validación con JavaScript:

```
if (nombre == "") {
   mensaje.textContent = "Debes escribir tu nombre";
} else {
   mensaje.textContent = "Datos correctos";
}
```

Aquí podemos decidir qué mensaje mostrar dependiendo de la condición que se cumpla.

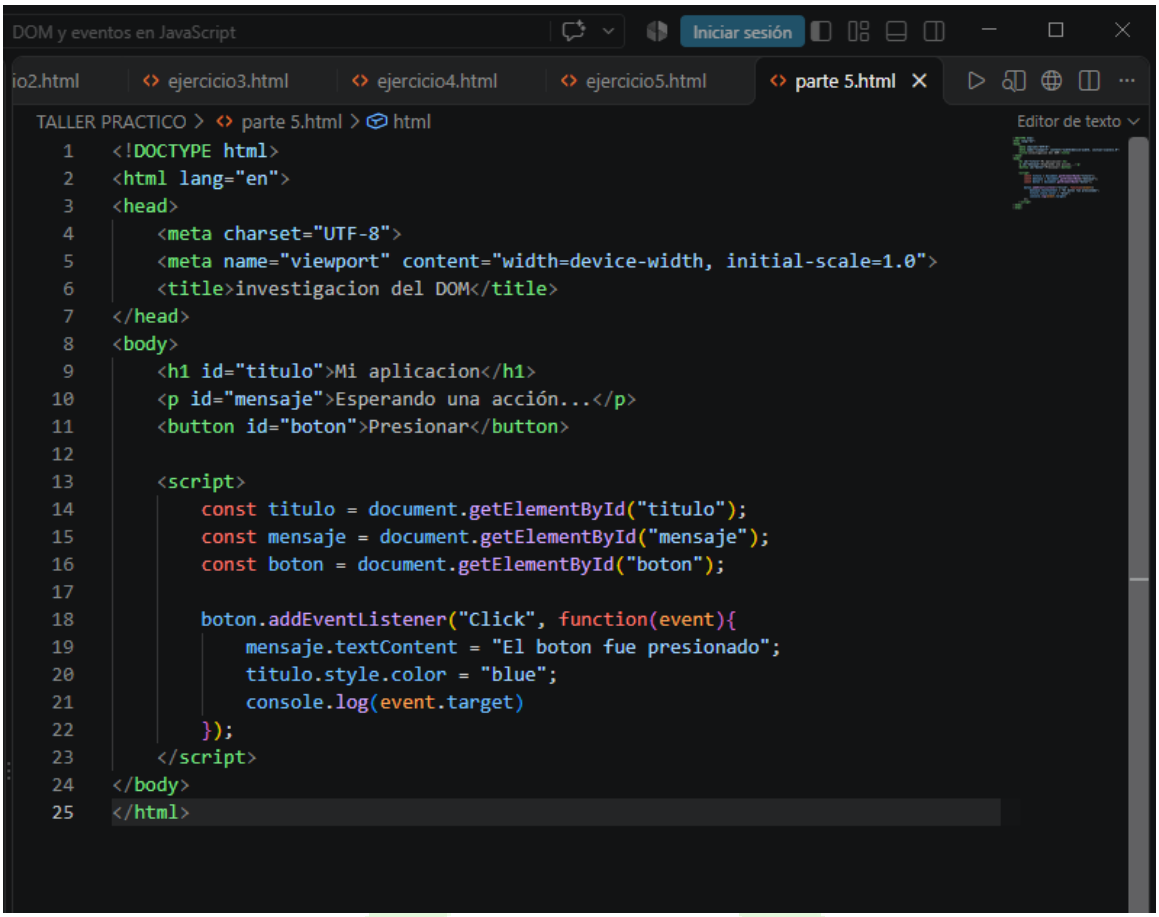
En conclusión

- submit: detecta el intento de envío del formulario.
- preventDefault(): evita el comportamiento predeterminado del navegador.
- Validación HTML: permite hacer comprobaciones básicas fácilmente.

- Validación JavaScript: permite crear comprobaciones y mensajes personalizados.

Lo mejor es combinar ambas validaciones cuando sea necesario. Para una aplicación real, también es importante validar los datos en el servidor, porque las comprobaciones del navegador se pueden evitar.

Parte 5. Análisis de código



27. ¿Qué elementos se seleccionan mediante getElementById()?

Se seleccionan tres elementos HTML mediante su identificador:

- titulo: el encabezado <h1>.
- mensaje: el párrafo <p>.
- boton: el botón <button>.

28. ¿Qué función cumple addEventListener() en este código?

Permite detectar cuándo ocurre un evento. En este caso, detecta cuando se presiona el botón y ejecuta una función que cambia el texto, modifica el color del título y muestra información en la consola.

29. ¿Qué sucedería si se reemplaza click por dblclick?

La función se ejecutaría cuando el usuario haga doble clic en el botón, en lugar de ejecutarse con un solo clic.

30. ¿Qué cambia cuando se ejecuta la función?

El párrafo cambia su texto a «El boton fue presionado», el título cambia a color azul y la consola muestra información del botón que generó el evento.

31. ¿Qué información muestra console.log(event.target)?

Muestra en la consola el elemento HTML que originó el evento. En este caso, muestra el botón que fue presionado.

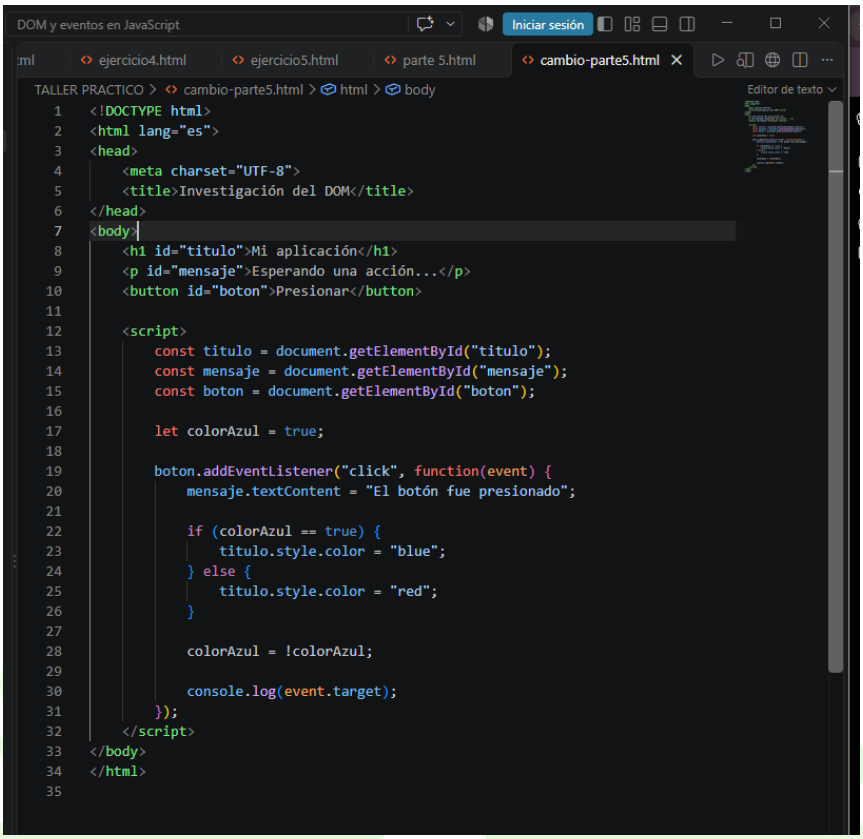
32. ¿Qué pasaría si el identificador del botón fuera incorrecto?

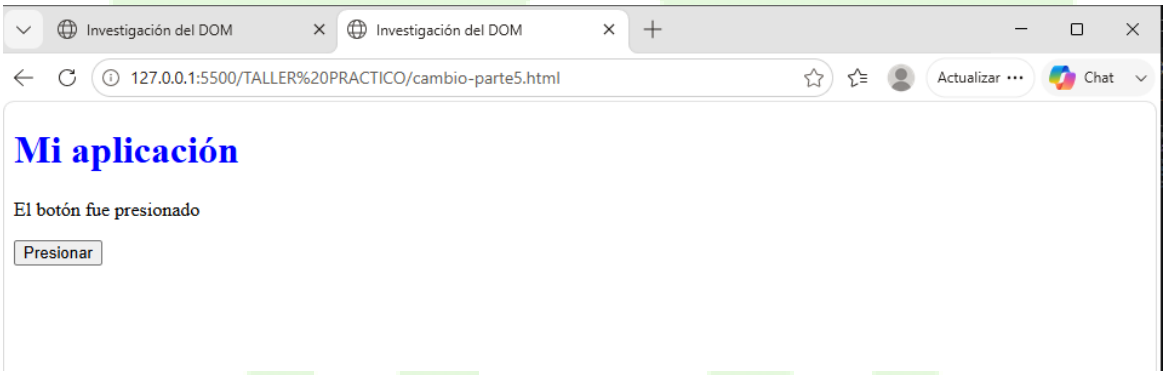
getElementById() devolvería null, porque no encontraría ningún elemento con ese identificador. Al intentar utilizar addEventListener() sobre ese valor, JavaScript produciría un error.

33. ¿Qué errores podrían ocurrir si se intenta acceder a un elemento que no existe?

JavaScript puede devolver null al intentar seleccionar el elemento. Si después intentamos acceder a propiedades como textContent, style o métodos como addEventListener(), se producirá un error porque el elemento no existe.

34. Modifique el código para que el título cambie de color cada vez que se presione el botón, alternando entre azul y rojo.





Explicación: La variable colorAzul comienza en true. Si es verdadera, el título cambia a azul; de lo contrario, cambia a rojo. La instrucción colorAzul = !colorAzul invierte su valor para que el siguiente clic cambie al otro color.

35. Explique cómo comprobaría que el evento funciona sin depender únicamente de la apariencia visual.

Abriría las herramientas de desarrollador del navegador con F12 y entraría en la pestaña Console. Después, presionaría el botón y comprobaría que aparece el elemento HTML del botón en la consola. También verificaría que el texto del párrafo cambia correctamente.

36. Proponga una mejora para separar el código JavaScript del documento HTML.

Una mejora sería crear un archivo independiente llamado script.js y trasladar allí el código JavaScript. En el HTML se agregaría esta línea:

<script src="script.js" defer></script>

Esto permite mantener el HTML y JavaScript separados, haciendo que el código sea más ordenado, fácil de entender y sencillo de modificar. El atributo defer permite que el navegador analice primero el HTML antes de ejecutar el archivo JavaScript.

Parte 6. Investigación aplicada a un caso real

1. Nombre y propósito del sistema

Nombre del sistema: Plataforma educativa virtual.

Propósito: Permitir que los estudiantes consulten sus cursos, busquen actividades y entreguen trabajos desde una plataforma digital, facilitando el aprendizaje y la comunicación con los docentes.

2. Tres elementos de la interfaz, eventos y acciones de JavaScript

Elemento de la interfaz	Evento	Acción de JavaScript
Botón «Consultar cursos»	click	Mostrar la lista de cursos disponibles.
Campo de búsqueda	input	Mostrar los cursos que coincidan con el texto escrito.
Formulario de entrega de trabajos	submit	Comprobar que los campos estén completos y mostrar un mensaje de confirmación.

3. ¿Cómo se relaciona esta interacción con el DOM?

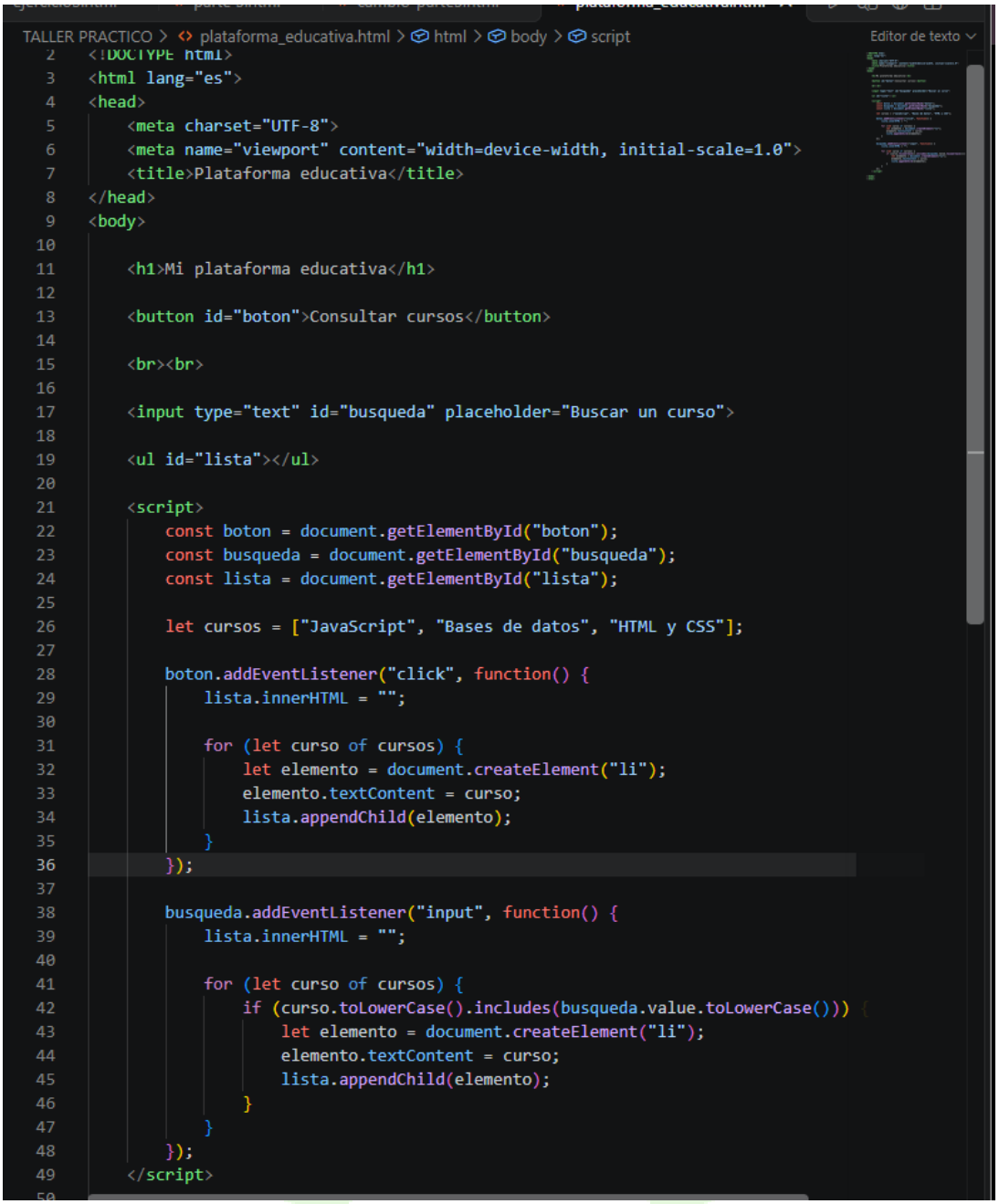
El DOM permite que JavaScript acceda a los elementos HTML de la plataforma y los modifique cuando el usuario interactúa con ellos.

Por ejemplo, cuando el estudiante presiona el botón «Consultar cursos», JavaScript detecta el evento click y puede mostrar los cursos en la página sin tener que modificar manualmente el HTML.

De esta manera, el DOM permite crear páginas web interactivas y dinámicas.

4. Pequeño prototipo HTML

Este prototipo representa una plataforma educativa sencilla en la que el estudiante puede consultar sus cursos y buscar uno por su nombre.



5. Explicación del prototipo

- getElementById() selecciona los elementos HTML que se van a utilizar.
- addEventListener("click", ...) detecta cuándo se presiona el botón para consultar los cursos.
- addEventListener("input", ...) detecta los cambios en el campo de búsqueda mientras el estudiante escribe.
- createElement("li") crea un elemento de lista para mostrar cada curso.
- appendChild() agrega los cursos a la lista de la página.
- innerHTML = "" limpia la lista antes de mostrar los resultados nuevos.
- includes() comprueba si el nombre del curso contiene el texto buscado.

- toLowerCase() permite comparar los nombres sin distinguir entre mayúsculas y minúsculas.

6. Conclusión

El DOM y los eventos de JavaScript permiten que una plataforma educativa responda a las acciones del usuario. Gracias a ellos, se pueden consultar cursos, buscar información y actualizar el contenido de una página de forma dinámica, mejorando la experiencia de los estudiantes.